

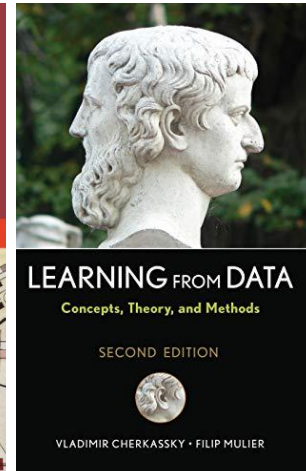
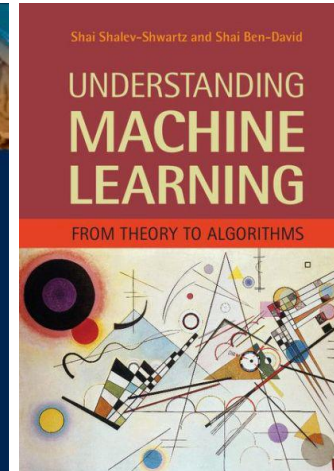
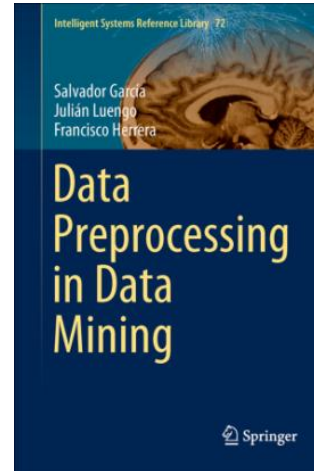
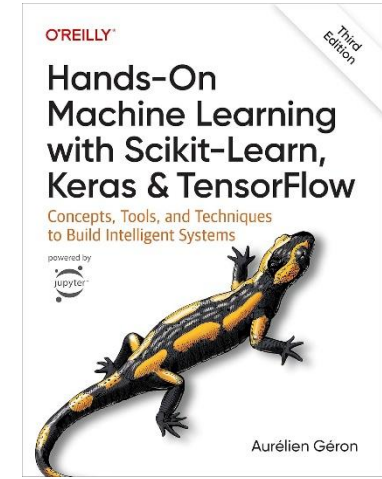
Aprendizaje Automático

Curso 2025-2026

Tema 4. Aprendizaje no supervisado



Bibliografía



- 
- Tema 1: Conceptos básicos del Aprendizaje Automático
 - Tema 2: Aprendizaje Supervisado. Regresión y Clasificación. Modelos. Evaluación
 - Tema 3: Ingeniería de Características y Preprocesamiento de Datos. Calidad de Datos
 - **Tema 4: Aprendizaje no Supervisado**
 - Tema 5: Aprendizaje Profundo. Fundamentos
 - Tema 6: Aprendizaje Profundo en diferentes tipos de datos y problemas
 - Tema 7. Aprendizaje por refuerzo
 - Tema 8: Optimización en Aprendizaje Automático. Hiperparámetros, selección de modelos y regularización
 - Tema 9: Aspectos Avanzados, retos y desafíos del aprendizaje automático

Fundamentos Teóricos

Técnicas de aprendizaje no supervisado

- Clustering
 - El objetivo es agrupar instancias similares en clusters. El clustering es una gran herramienta para el análisis de datos, la segmentación de clientes, los sistemas de recomendación, los motores de búsqueda, la segmentación de imágenes, el aprendizaje semisupervisado, la reducción de dimensionalidad, etc.
- Detección de anomalías y valores atípicos
 - El objetivo es aprender qué aspecto tienen los datos "normales" y, a partir de ahí, detectar los casos anómalos. Estos casos se denominan anomalías, mientras que los casos normales se denominan "inliers". La detección de anomalías es útil en una amplia variedad de aplicaciones, como la detección de fraudes, la detección de productos defectuosos en la fabricación, la identificación de nuevas tendencias en series temporales o la eliminación de valores atípicos de un conjunto de datos antes de entrenar otro modelo, lo que puede mejorar significativamente el rendimiento del modelo resultante.

Fundamentos Teóricos

Técnicas de aprendizaje no supervisado

- Estimación de la densidad
 - Se trata de estimar la *función de densidad de probabilidad* (PDF) del proceso aleatorio que ha generado el conjunto de datos. La estimación de la densidad se utiliza habitualmente para la detección de anomalías: es probable que los casos situados en regiones de muy baja densidad sean anomalías. También es útil para el análisis y la visualización de datos.

Clustering

- Tarea de identificar instancias similares y asignarlas a *clusters*, o grupos de instancias similares.
- Al igual que en la clasificación, cada instancia se asigna a un grupo. Sin embargo, a diferencia de la clasificación, el clustering es una tarea no supervisada.

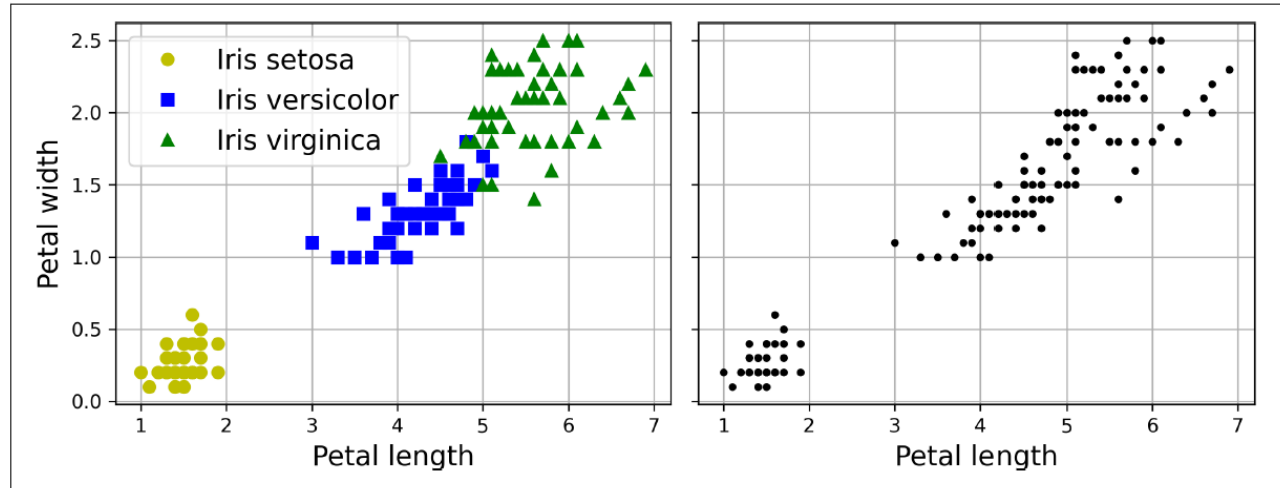


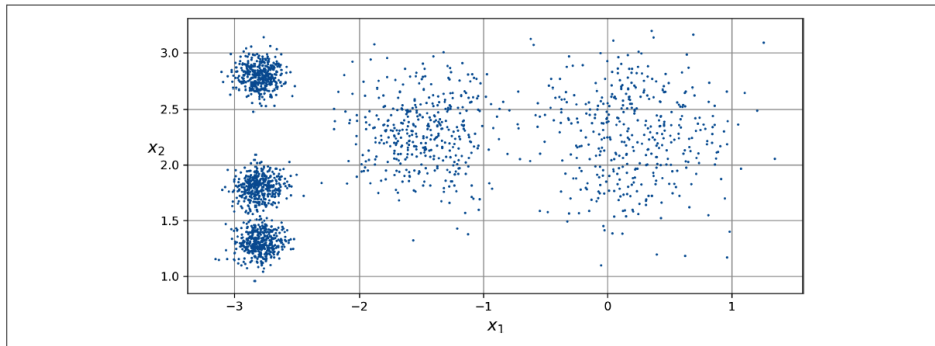
Figure 9-1. Classification (left) versus clustering (right)

Aplicaciones del clustering

- Segmentación de clientes
- Análisis de datos
- Reducción de la dimensionalidad
 - Una vez que se ha agrupado un conjunto de datos, suele ser posible medir la *afinidad* de cada instancia con cada clúster; la afinidad es cualquier medida de lo bien que encaja una instancia en un clúster. El vector de características $\mathbf{x}$ de cada instancia puede sustituirse por el vector de sus afinidades con los clusters. Si hay k conglomerados, este vector es *k-dimensional*.
- Ingeniería de características (feature engineering)
- Detección de anomalías
- Aprendizaje semisupervisado
- Motores de búsqueda
- Segmentación de imágenes
 - Agrupando los píxeles según su color y sustituyendo después el color de cada píxel por el color medio de su agrupación, es posible reducir considerablemente el número de colores diferentes de una imagen.

k-Means

- Considera el conjunto de datos sin etiquetar representado aquí: puedes ver claramente cinco manchas de instancias.
- El algoritmo k-means es un algoritmo sencillo capaz de agrupar este tipo de conjuntos de datos de forma muy rápida y eficaz, a menudo en sólo unas pocas iteraciones.
- Fue propuesto por Stuart Lloyd en los laboratorios Bell en 1957 como técnica para la modulación por impulsos codificados, pero no se publicó fuera de la empresa hasta 1982. En 1965, Edward W. Forgy había publicado prácticamente el mismo algoritmo, por lo que a veces se hace referencia a k-means como el algoritmo Lloyd-Forgy.

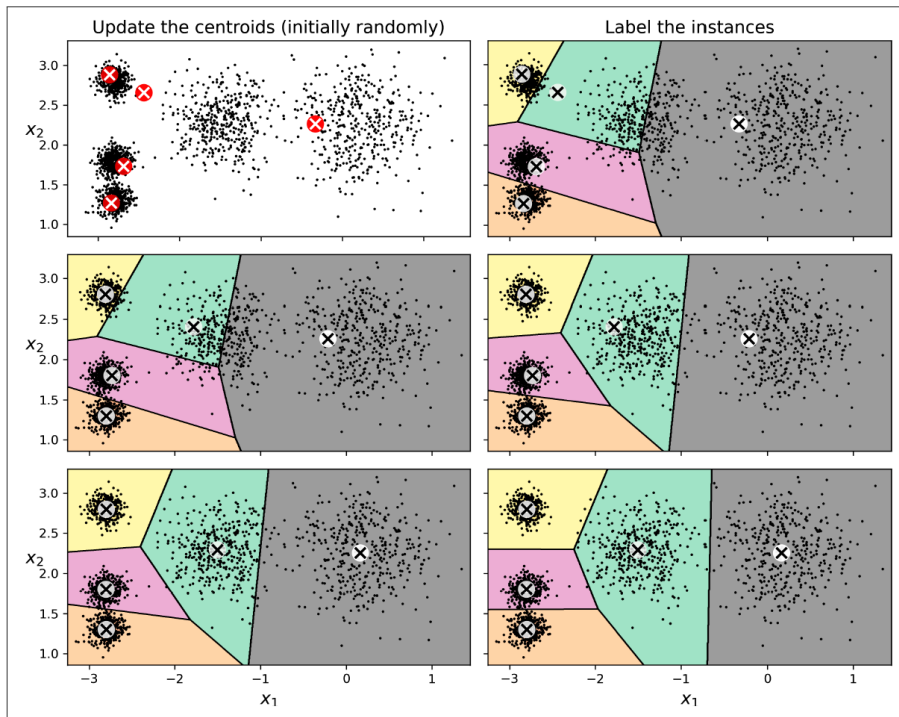


k-Means

- El algoritmo k-means es uno de los algoritmos de clustering más rápidos y también uno de los más sencillos.
- En primer lugar, se inicializan k centroides aleatoriamente: por ejemplo, se eligen al azar k instancias distintas del conjunto de datos y se colocan los centroides en sus ubicaciones.
- Repita la operación hasta la convergencia (es decir, hasta que los centroides dejen de moverse):
 - Asignar cada instancia al centroide más cercano.
 - Actualiza los centroides para que sean la media de las instancias que se les asignan.
- La clase `KMeans` utiliza por defecto una técnica de inicialización optimizada. Para obtener el algoritmo original de K-Means, debe establecer `init="random"` y `n_init=1`.
- La complejidad computacional del algoritmo suele ser lineal con respecto al número de instancias m , el número de conglomerados k y el número de dimensiones n . Sin embargo, esto sólo es cierto cuando los datos tienen una estructura de conglomerados. Si no es así, en el peor de los casos la complejidad puede aumentar exponencialmente con el número de instancias. En la práctica, esto rara vez ocurre, y k-means suele ser uno de los algoritmos de clustering más rápidos.

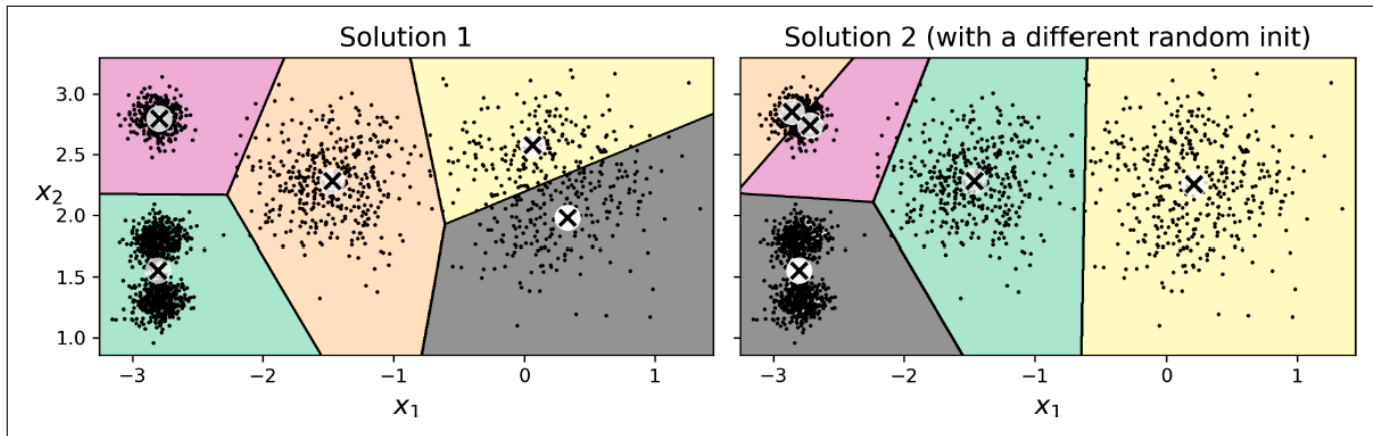
k-Means

- Ejecución del algoritmo k-means durante 1, 2 y 3 iteraciones, para ver cómo se mueven los centroides:



k-Means

- Aunque se garantiza que el algoritmo converge, puede que no converja a la solución correcta (es decir, puede que converja a un óptimo local): que lo haga o no depende de la inicialización del centroide.
- La siguiente figura muestra dos soluciones subóptimas a las que puede converger el algoritmo si no se tiene suerte con el paso de inicialización aleatoria.



k-means++

- Hay un paso de inicialización más inteligente que tiende a seleccionar centroides distantes entre sí, y esta mejora hace que el algoritmo k-means tenga muchas menos probabilidades de converger a una solución subóptima.
- El paper propuesta demostró que el cálculo adicional necesario para el paso de inicialización más inteligente merece la pena porque permite reducir drásticamente el número de veces que hay que ejecutar el algoritmo para encontrar la solución óptima.

<https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf>

k-means++

El algoritmo de inicialización *de k-means++* funciona así:

1. Toma un centroide $\mathbf{c}^{(1)}$, elegido uniformemente al azar del conjunto de datos.
2. Toma un nuevo centroide $\mathbf{c}^{(i)}$, eligiendo una instancia $\mathbf{x}^{(i)}$ con probabilidad

$$D(\mathbf{x}^{(i)})^2 / \sum_{j=1}^m D(\mathbf{x}^{(j)})^2$$

donde $D(\mathbf{x}^{(i)})$ es la distancia entre la instancia $\mathbf{x}^{(i)}$ y el centroide más cercano ya elegido. Esta distribución de probabilidad garantiza que las instancias más alejadas de los centroides ya elegidos tengan muchas más probabilidades de ser seleccionadas como centroides.

3. Repita el paso anterior hasta que se hayan elegido los k centroides.

- La clase *KMeans* de scikit-learn utiliza este método de inicialización por defecto.

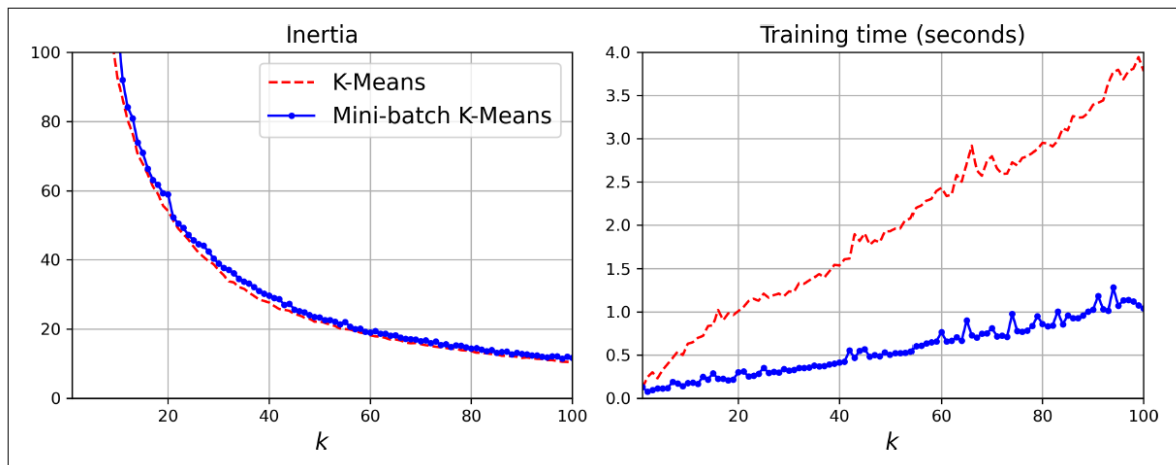
k-means acelerado

- Charles Elkan propuso en 2003 otra mejora del algoritmo k-means.
- En algunos conjuntos de datos grandes con muchos clusters, el algoritmo puede acelerarse evitando muchos cálculos de distancia innecesarios.
- Elkan lo consiguió explotando la desigualdad del triángulo (es decir, que una línea recta es siempre la distancia más corta entre dos puntos) y llevando un registro de los límites inferior y superior de las distancias entre instancias y centroides.
- Sin embargo, el algoritmo de Elkan no siempre acelera el entrenamiento, y a veces incluso puede ralentizarlo significativamente; depende del conjunto de datos.
- Aun así, si quieres probarlo, pon `algorithm="elkan"` como hiperparámetro.

<https://cdn.aaai.org/ICML/2003/ICML03-022.pdf>

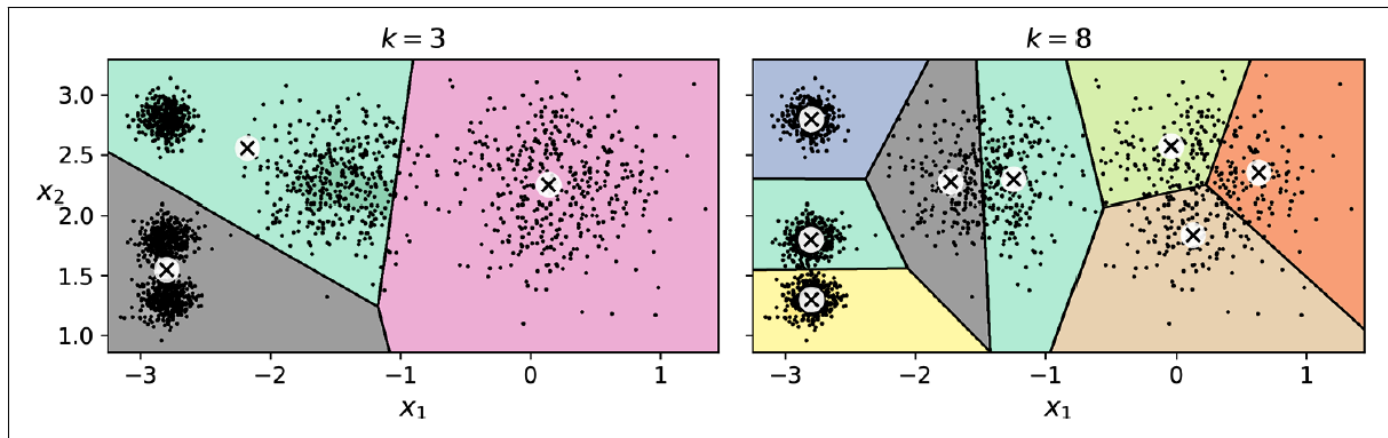
Mini-Batch k-means

- En lugar de utilizar el conjunto de datos completo en cada iteración, el algoritmo es capaz de utilizar minilotes, moviendo los centroides sólo ligeramente en cada iteración.
- Esto acelera el algoritmo (normalmente por un factor de tres a cuatro) y permite agrupar conjuntos de datos enormes que no caben en la memoria.
- Aunque el algoritmo mini-batch k-means es mucho más rápido que el algoritmo k-means normal, su inercia es en general ligeramente peor. Esto se puede ver aquí.



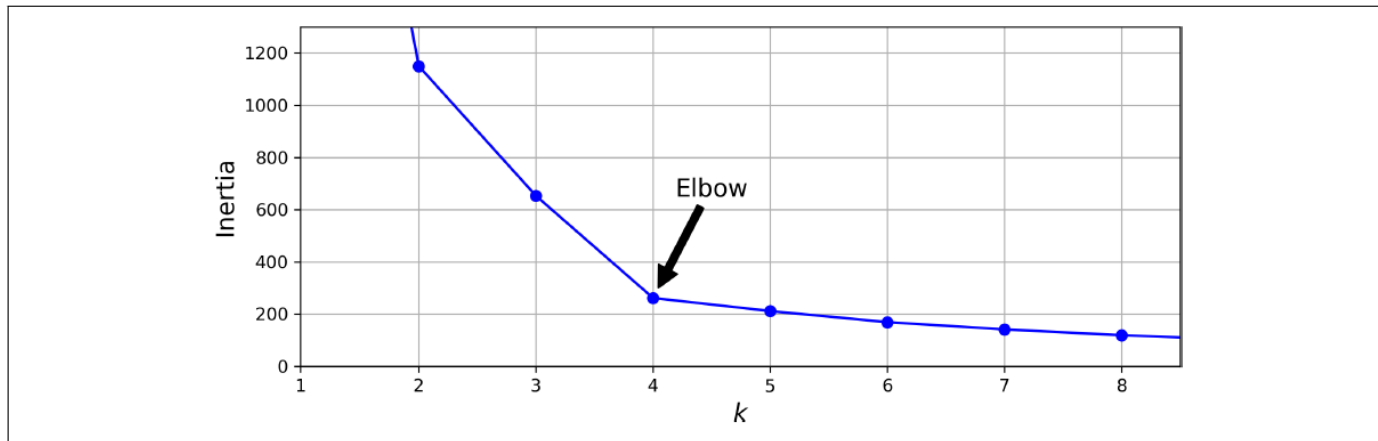
Encontrar el número óptimo de clusters

En general, no será tan fácil saber cómo establecer k , y el resultado puede ser bastante malo si lo establece en el valor incorrecto. Como se puede ver aquí, para este conjunto de datos establecer k en 3 u 8 da como resultado modelos bastante malos.



Encontrar el número óptimo de clusters

- La inercia es la suma de las distancias al cuadrado entre las instancias y sus centroides más cercanos.
- La inercia no es una buena métrica de rendimiento cuando se trata de elegir k porque sigue bajando a medida que aumentamos k .
- En efecto, cuantos más conglomerados haya, más cerca estará cada instancia de su centroide más próximo y, por tanto, menor será la inercia. Tracemos la inercia en función de k . Al hacerlo, la curva suele contener un punto de inflexión llamado *codo*.

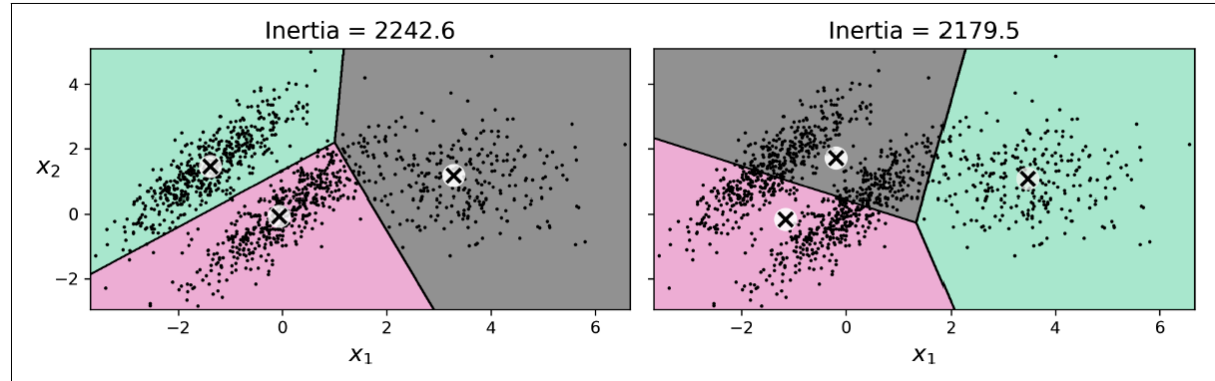


Encontrar el número óptimo de clusters

- La técnica del codo para elegir el mejor valor del número de conglomerados es bastante tosca.
- Un enfoque más preciso (pero también más costoso desde el punto de vista informático) es utilizar la *puntuación de silueta*, que es el *coeficiente de silueta* medio de todas las instancias.
- El coeficiente de silueta de una instancia es igual a $(b - a) / \max(a, b)$, donde a es la distancia media a las demás instancias del mismo clúster (es decir, la distancia media intraclúster) y b es la distancia media clúster más cercano (es decir, la distancia media a las instancias del clúster más cercano, definido como aquel que minimiza b , excluyendo el clúster propio de la instancia).
- El coeficiente de silueta puede variar entre -1 y +1. Un coeficiente cercano a +1 significa que la instancia está bien dentro de su propio clúster y lejos de otros clústeres, mientras que un coeficiente cercano a 0 significa que está cerca del límite de un clúster; por último, un coeficiente cercano a -1 significa que la instancia puede haber sido asignada a un clúster equivocado.

Límites de k-Means

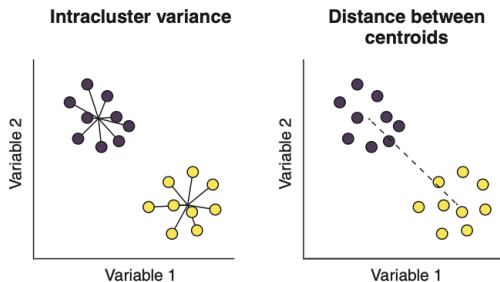
- k-Means no es perfecto. Es necesario ejecutar el algoritmo varias veces para evitar soluciones subóptimas, y además hay que especificar el número de clusters. Además, k-Means no se comporta muy bien cuando los clusters tienen tamaños variables, densidades diferentes o formas no esféricas.
- Aquí mostramos cómo k-Means agrupa un conjunto de datos que contiene tres clusters elipsoidales de diferentes dimensiones, densidades y orientaciones. Ninguna de estas soluciones es buena. La solución de la izquierda es mejor, pero elimina el 25% del cluster central y lo asigna al cluster de la derecha. La solución de la derecha es terrible, aunque su inercia sea menor.



Medidas de calidad del clustering

Davies-Boulding index

- ❑ Cuantifica la "separabilidad media" de un clúster frente a la del clúster más cercano
- ❑ **Cálculo:** varianza dentro de los clústeres (dispersión o *scatter*) / separación entre centroides
- ❑ Mejor cuanto más pequeño



$$\text{scatter}_k = \left(\frac{1}{n_k} \sum_{i \in k}^{n_k} (x_i - c_k)^2 \right)^{1/2}$$

$$\text{separation}_{j,k} = \left(\sum_{1 \leq j \leq k}^N (c_j - c_k)^2 \right)^{1/2}$$

$$\text{ratio}_{j,k} = \frac{\text{scatter}_j + \text{scatter}_k}{\text{separation}_{j,k}}$$

$$\text{DB} = \frac{1}{N} \sum_{k=1}^N R_k$$

R_k es el máximo $\text{ratio}_{j,k}$

Medidas de calidad del clustering

• Silueta

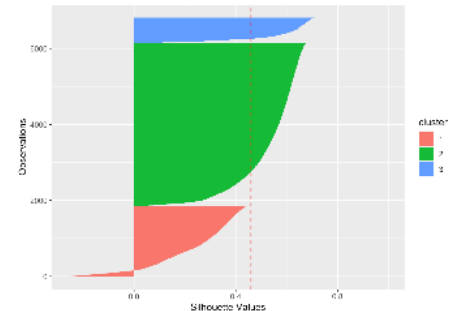
- ❑ Cuantifica cómo de lejano es un objeto de un clúster a los objetos de los otros clústeres; esto es, si está separado del borde o no
- ❑ **Cálculo:** para cada objeto, se suma por un lado la distancia con todos los objetos de su clúster (a) y por otro la distancia con los objetos del clúster más cercano (b); finalmente, se calcula el ratio entre ambas
- ❑ En $[-1, 1]$, mejor cuanto más próximo a 1

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d(i, j)$$

$$b(i) = \min_{k \neq i} \frac{1}{|C_k|} \sum_{j \in C_k} d(i, j)$$

$$s(i) = \begin{cases} 1 - a(i)/b(i), & \text{if } a(i) < b(i) \\ 0, & \text{if } a(i) = b(i) \\ b(i)/a(i) - 1, & \text{if } a(i) > b(i) \end{cases}$$

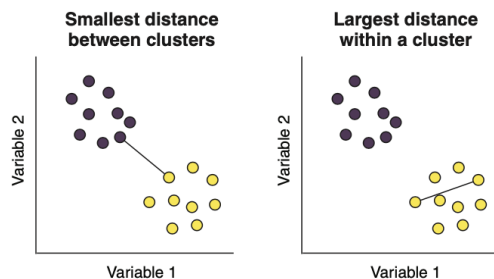
Buscamos formas homogéneas y que estén por encima de la media



Medidas de calidad del clustering

Dunn index

- ❑ Cuantifica el ratio entre la distancia mínima entre puntos de diferentes clústeres y el diámetro máximo de los clústeres
- ❑ Mejor cuanto más alto



H.I. Rhys. *Machine Learning with R, the tidyverse, and mlr*. Manning, 2020.

$$\text{Dunn} = \min_{1 \leq i \leq k} \left\{ \min_k \left(\frac{\delta(c_i, c_j)}{\max_{1 \leq i \neq j \leq k} \Delta(c_k)} \right) \right\}$$

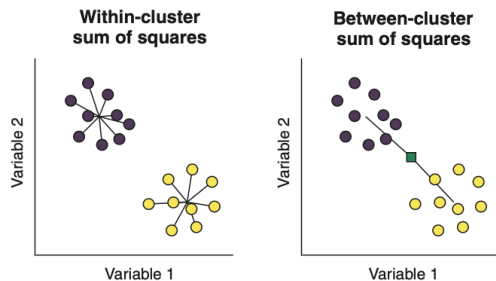
$\delta(c_i, c_j)$ es la distancia entre todos los pares de los clústeres i, j

$\Delta(c_k)$ es la distancia máxima entre objetos del clúster k

Medidas de calidad del clustering

Estadística pseudo F

- ❑ Cuantifica el ratio entre las distancias "entre-cluster" (distancia del centroide del clúster al super-centroide ponderada por el número de objetos en el clúster) y la distancia "intra-cluster" (distancia entre cada objeto de un clúster y el centroide)
- ❑ Mejor cuanto más alto



$$\text{Pseudo F} = \frac{SS_{\text{between}} / (k - 1)}{SS_{\text{within}} / (n - k)}$$

where SS_{between} and SS_{within} are calculated as

$$SS_{\text{between}} = \sum_k^N n_k (c_k - c_g)^2$$

$$SS_{\text{within}} = \sum_k^N \sum_{i \in k}^{n_k} (x_i - c_k)^2$$

N es el número de clústeres, n_k es el número de objetos en el clúster k , c_k es el centroide de k , c_g es el super-centroide

DBSCAN

- El algoritmo de *agrupación espacial de aplicaciones con ruido basado en la densidad* (DBSCAN) define los clusters como regiones continuas de alta densidad. Funciona de la siguiente manera:
 - Para cada instancia, el algoritmo cuenta cuántas instancias se encuentran a una pequeña distancia ϵ (épsilon) de ella. Esta región se denomina *vecindad ϵ* de la instancia.
 - Si una instancia tiene al menos *min_muestras de* instancias en su vecindad ϵ (incluida ella misma), se considera una *instancia central*. En otras palabras, las instancias centrales son aquellas que se encuentran en regiones densas.
 - Todas las instancias en la vecindad de una instancia núcleo pertenecen al mismo cluster. Esta vecindad puede incluir otras instancias de núcleo; por lo tanto, una secuencia larga de instancias de núcleo vecinas forma un único clúster.
 - Cualquier instancia que no sea una instancia central y no tenga ninguna en su vecindad se considera una anomalía.
- Este algoritmo funciona bien si todos los clusters están bien separados por regiones de baja densidad.

DBSCAN

- DBSCAN es un algoritmo muy sencillo pero potente, capaz de identificar cualquier número de conglomerados de cualquier forma. Es robusto frente a valores atípicos y solo tiene dos hiperparámetros (`eps` y `min_samples`).
- Sin embargo, si la densidad varía significativamente entre los conglomerados, o si no hay una región de densidad suficientemente baja alrededor de algunos conglomerados, DBSCAN puede tener dificultades para capturar todos los conglomerados correctamente.
- Además, su complejidad computacional es aproximadamente $O(m^2 n)$, por lo que no se adapta bien a grandes conjuntos de datos.
- También puede probar DBSCAN jerárquico (HDBSCAN), que se implementa en el proyecto `scikit-learn-contrib`, ya que suele ser mejor que DBSCAN a la hora de encontrar clústeres de distintas densidades.

Otros algoritmos de clustering

- Clustering aglomerativo
 - Se construye una jerarquía de clusters de abajo a arriba. En cada iteración, la agrupación aglomerativa conecta el par de clusters más cercano (empezando por instancias individuales). Si dibujáramos un árbol con una rama por cada par de clusters que se fusionaran, obtendríamos un árbol binario de clusters, donde las hojas son las instancias individuales. Este enfoque puede capturar clusters de diversas formas; también produce un árbol de clusters flexible e informativo en lugar de obligarle a elegir una escala de conglomerados concreta, y puede utilizarse con cualquier distancia entre pares. Puede escalar bien a un gran número de instancias si se proporciona una matriz de conectividad, que es una matriz $m \times m$ dispersa que indica qué pares de instancias son vecinos (por ejemplo, devuelta por [sklearn.neighbors.kneighbors_graph\(\)](#)). Sin una matriz de conectividad, el algoritmo no se adapta bien a grandes conjuntos de datos.
- BIRCH
 - El algoritmo BIRCH (balanced iterative reducing and clustering using hierarchies) se diseñó específicamente para conjuntos de datos muy grandes, y puede ser más rápido que k-means por lotes, con resultados similares, siempre que el número de características no sea demasiado grande (<20). Durante el entrenamiento, construye una estructura de árbol que contiene la información suficiente para asignar rápidamente cada nueva instancia a un clúster, sin tener que almacenar todas las instancias en el árbol: este enfoque le permite utilizar una memoria limitada mientras maneja enormes conjuntos de datos.

Otros algoritmos de clustering

- Mean-shift
 - Este algoritmo comienza colocando un círculo centrado en cada instancia; luego, para cada círculo, calcula la media de todas las instancias situadas dentro de él y desplaza el círculo de modo que quede centrado en la media. A continuación, itera este paso de desplazamiento de la media hasta que todos los círculos dejan de moverse (es decir, hasta que cada uno de ellos está centrado en la media de las instancias que contiene). El desplazamiento de la media desplaza los círculos en la dirección de mayor densidad, hasta que cada uno de ellos haya encontrado un máximo local de densidad. Por último, todas las instancias cuyos círculos se han asentado en el mismo lugar (o lo suficientemente cerca) se asignan al mismo cluster.
 - Mean-shift tiene algunas de las mismas características que DBSCAN, como que puede encontrar cualquier número de clusters de cualquier forma, tiene muy pocos hiperparámetros (sólo uno: el radio de los círculos, llamado **bandwidth**) y se basa en la estimación de la densidad local. Pero, a diferencia del DBSCAN, el desplazamiento de la media tiende a trocear los clusters cuando presentan variaciones internas de densidad.
 - Por desgracia, su complejidad computacional es $O(m^2 n)$, por lo que no es adecuado para grandes conjuntos de datos.

Otros algoritmos de clustering

- Affinity Propagation

- En este algoritmo, las instancias intercambian mensajes entre sí repetidamente hasta que cada instancia ha elegido a otra instancia (o a sí misma) para que la represente. Estas instancias elegidas se denominan *exemplars*.
- Cada ejemplar y todas las instancias que lo eligieron forman un clúster. En la vida política real, uno suele querer votar a un candidato cuyas opiniones sean similares a las suyas, pero también quiere que gane las elecciones, por lo que puede elegir a un candidato con el que no esté totalmente de acuerdo, pero que sea más popular. La popularidad se evalúa normalmente a través de encuestas.
- La propagación por afinidad funciona de forma similar y tiende a elegir ejemplares situados cerca del centro de los conglomerados, de forma parecida a k-Means. Pero, a diferencia de k-means, no hay que elegir un número de clusters de antemano: se determina durante el entrenamiento.
- Además, la propagación por afinidad puede manejar muy bien clusters de diferentes tamaños.
- Lamentablemente, este algoritmo tiene una complejidad computacional de $O(m^2)$, por lo que no es adecuado para grandes conjuntos de datos.

Otros algoritmos de clustering

- Clustering espectral

- Este algoritmo toma una matriz de similitud entre las instancias y crea una incrustación (embedding) de baja dimensión a partir de ella (es decir, reduce la dimensionalidad de la matriz), luego utiliza otro algoritmo de clustering en este espacio de baja dimensión (la implementación de Scikit-Learn utiliza k-means).
- El clustering espectral puede capturar estructuras de clustering complejas, y también puede utilizarse para cortar grafos (por ejemplo, para identificar clusters de amigos en una red social).
- No se adapta bien a un gran número de instancias y no se comporta bien cuando los clusters tienen tamaños muy diferentes.

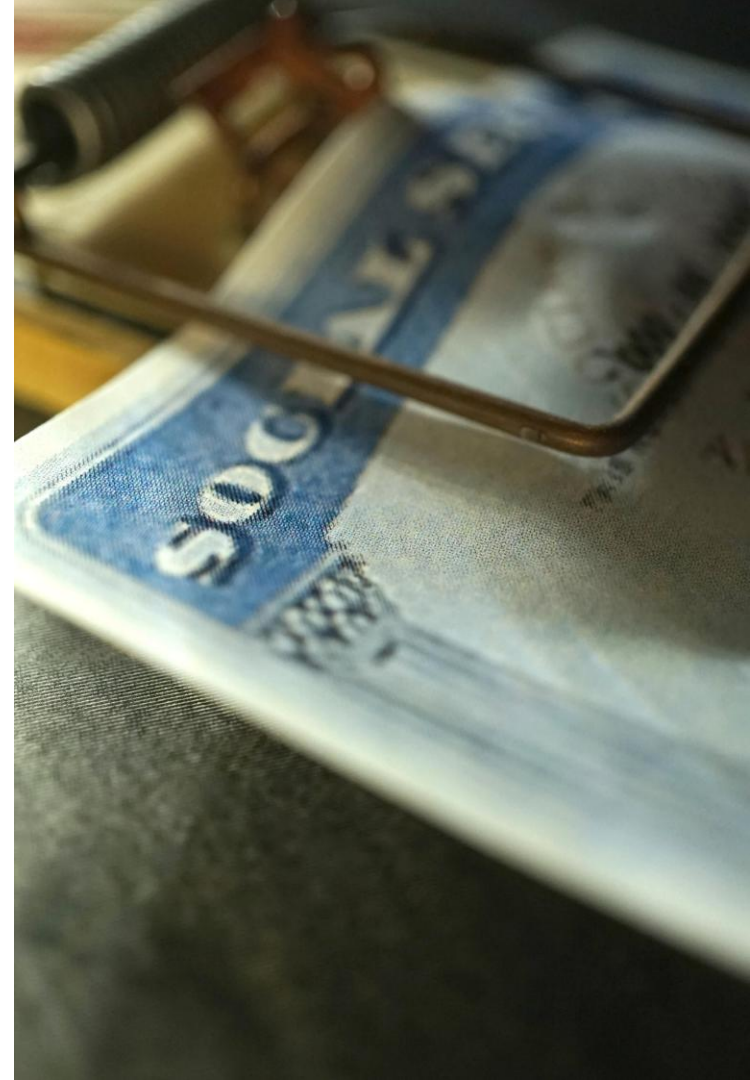
Detección de anomalías en Machine Learning

La detección de anomalías en Machine Learning es la identificación de patrones raros o no comunes en los datos. Las anomalías pueden indicar fallos en los dispositivos, fraudes, errores humanos, entre otros. Es una tarea importante en diferentes aplicaciones de Machine Learning.



APLICACIONES DE LA DETECCIÓN DE ANOMALÍAS

- Detección de fraude en tarjetas de crédito
- Identificación de fallos en dispositivos electrónicos
- Detección de intrusiones en sistemas de seguridad
- Identificación de enfermedades raras en el campo de la medicina



TÉCNICAS DE DETECCIÓN DE ANOMALÍAS

- Métodos basados en distancias (k-NN)
- Métodos basados en densidad (LOF)
- Métodos basados en modelos (OCSVM)
- Métodos basados en ensembles (IsolationForest)

Detección de anomalías

Key Challenges

- Definir una región normal representativa es un reto
- El límite entre el comportamiento normal y el atípico no suele ser preciso
- El concepto exacto de valor atípico varía según el ámbito de aplicación
- Disponibilidad de datos etiquetados para entrenamiento/validación
- Adversarios malintencionados
- Los datos pueden contener ruido
- El comportamiento normal sigue evolucionando

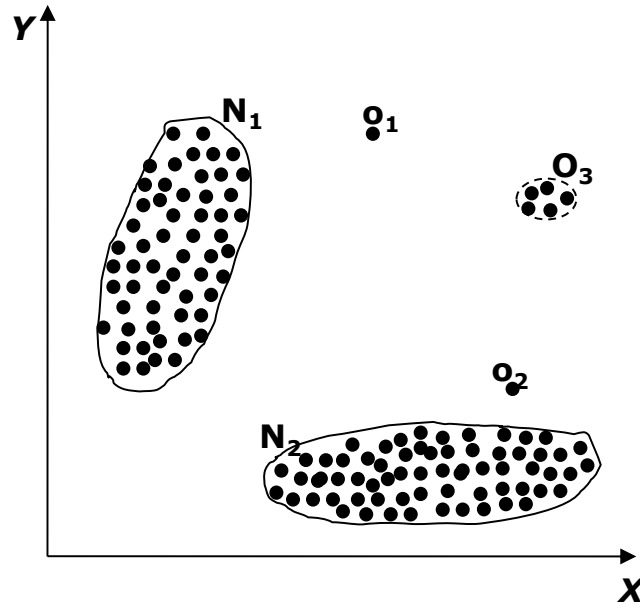


Detección de anomalías

¿qué son las anomalías?

Anomalías puntuales

- Una instancia de datos individual es anómala con respecto a los datos

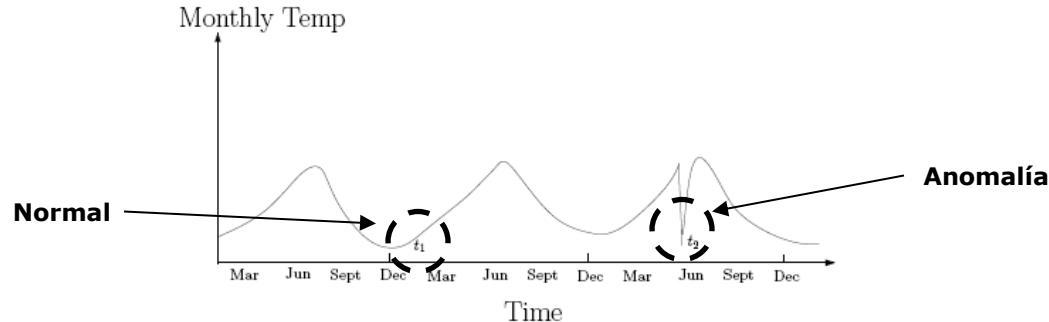


Detección de anomalías

¿qué son las anomalías?

Anomalías contextuales

- Un dato individual es anómalo dentro de un contexto
- Requiere una noción de contexto
- También se denominan anomalías condicionales *



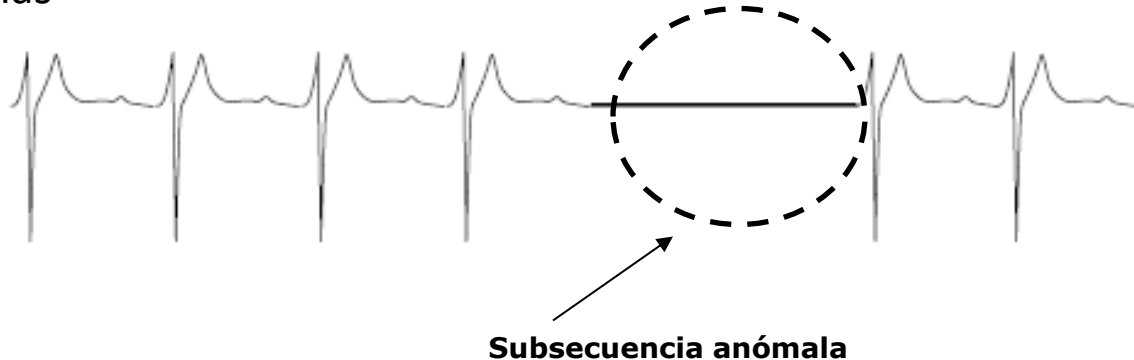
* Xiuyao Song, Mingxi Wu, Christopher Jermaine, Sanjay Ranka, Conditional Anomaly Detection, IEEE Transactions on Data and Knowledge Engineering, 2006.

Detección de anomalías

¿qué son las anomalías?

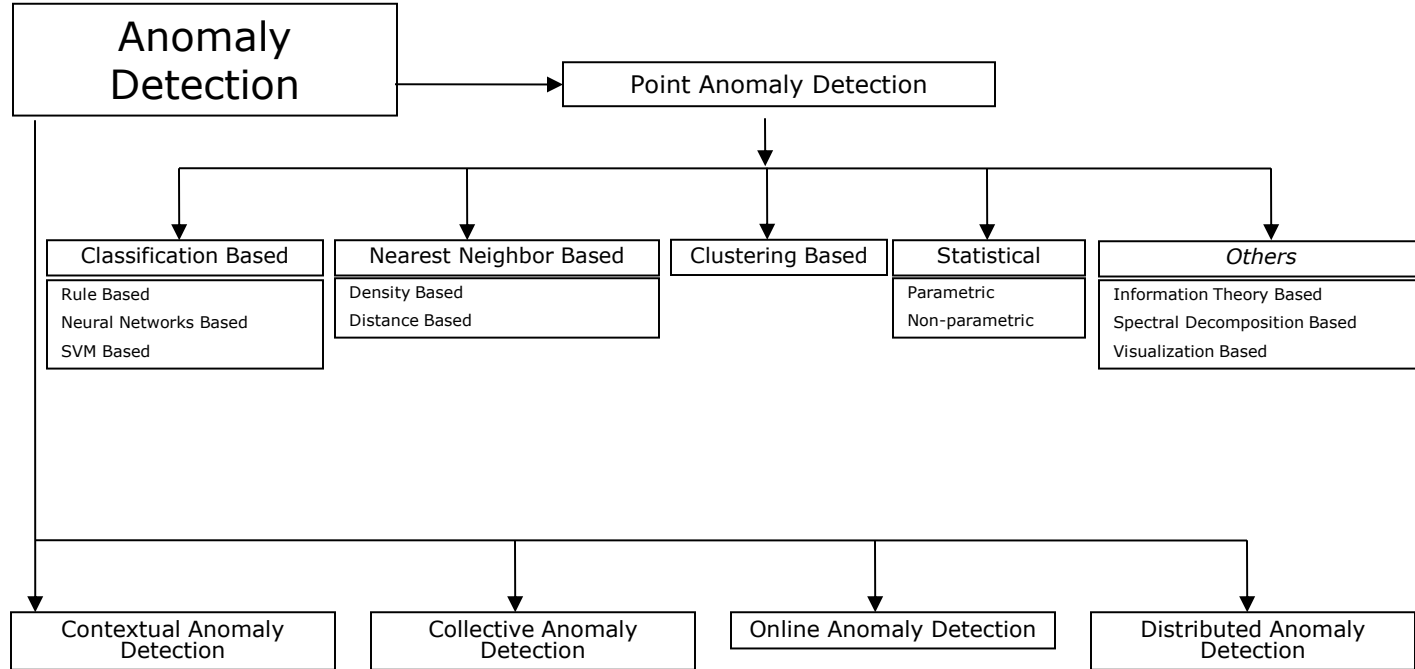
Anomalías colectivas

- Una colección de instancias de datos relacionadas es anómala
- Requiere una relación entre instancias de datos
- Datos secuenciales
- Datos espaciales
- Datos gráficos
- Las instancias individuales dentro de una anomalía colectiva no son anómalas por sí mismas



Detección de anomalías

Taxonomía



* Anomaly Detection – A Survey, Varun Chandola, Arindam Banerjee, and Vipin Kumar, ACM Computing Surveys, Vol. 41, No. 3, Article 15, Publication date: July 2009.

Detección de anomalías

Técnicas basadas en el vecino más próximo

- **Suposición clave:** los puntos normales tienen vecinos cercanos, mientras que las anomalías se sitúan lejos de otros puntos
- Enfoque general en dos pasos
 1. Calcula la vecindad de cada registro de datos
 2. Analiza la vecindad para determinar si el registro de datos es anómalo o no
- **Categorías:**
 - Métodos basados en la distancia
 - Las anomalías son los puntos de datos más alejados de otros puntos
 - Métodos basados en la densidad
 - Las anomalías son puntos de datos en regiones de baja densidad

Detección de anomalías

Técnicas basadas en el vecino más próximo

Detección de valores atípicos basada en la distancia

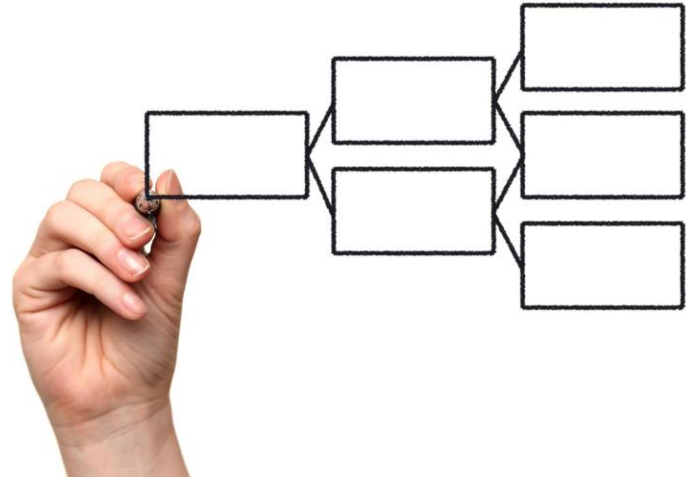
- *Enfoque del vecino más próximo (NN)*
 - Para cada punto de datos d , calcula la distancia al k -ésimo vecino más próximo d_k
 - Ordena todos los puntos de datos según la distancia d_k
 - Los valores atípicos son los puntos que tienen la mayor distancia d_k y, por tanto, están situados en los vecindarios más dispersos
 - Normalmente, los puntos de datos que tienen una distancia d_k superior al $n\%$ se identifican como valores atípicos
 - n – hiperparámetro
 - No es adecuado para conjuntos de datos que tienen modos con densidad variable

Detección de anomalías

Técnicas basadas en densidad

Funcionamiento del algoritmo Local Outlier Factor (LOF)

- El algoritmo LOF utiliza medidas de densidad local para detectar valores atípicos.
- Compara la densidad local de una instancia con la de sus vecinos.
- Si la densidad de una instancia es significativamente menor que la de sus vecinos, se considera un valor atípico.



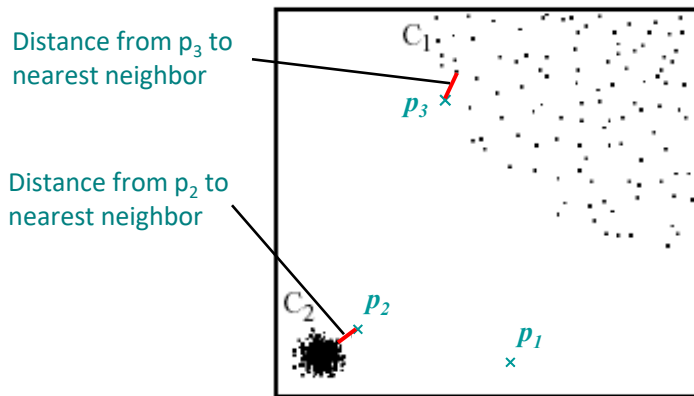
Detección de anomalías

Técnicas basadas en densidad

Ventajas de las técnicas basadas en la densidad

- *Local Outlier Factor (LOF)*

- Ejemplo:



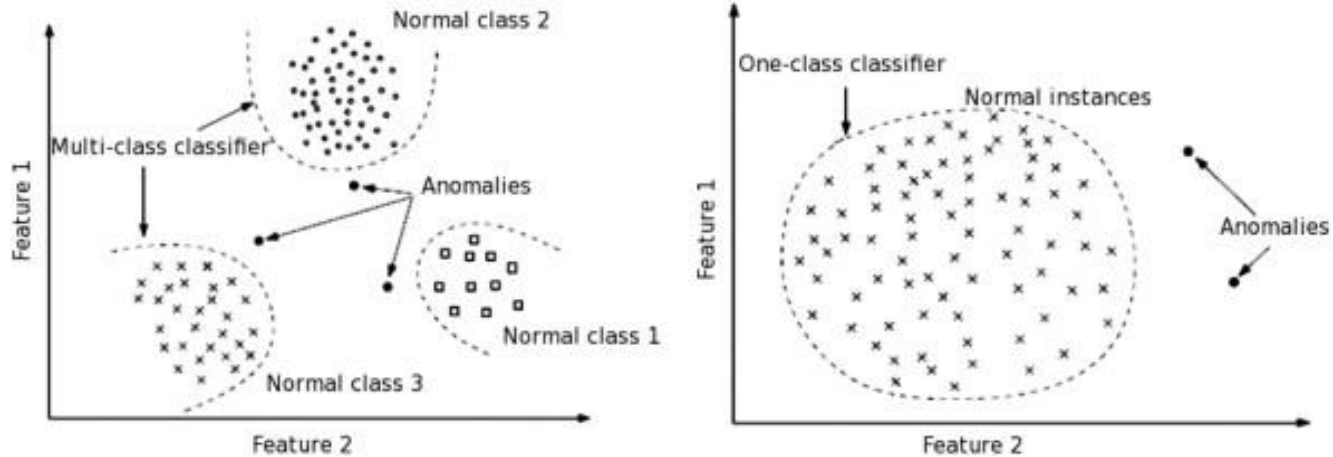
En el enfoque NN, p_2 no se considera un valor atípico, mientras que el enfoque LOF considera que tanto p_1 como p_2 son valores atípicos.

El enfoque NN puede considerar p_3 como valor atípico, pero el enfoque LOF no lo hace

Detección de anomalías

One-class Classification

Several classes vs One-class classification

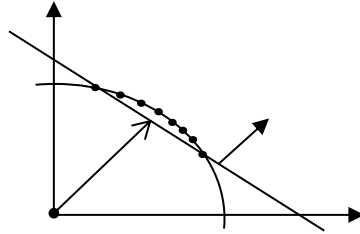


Detección de anomalías

One-class Classification

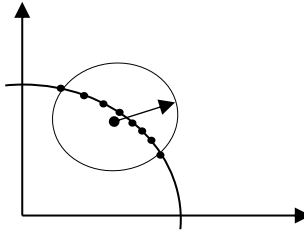
- One Class SVM

- Encuentra el **hiperplano óptimo** para separar la clase objetivo del origen con el máximo margen



- Descripción de los datos del vector de soporte

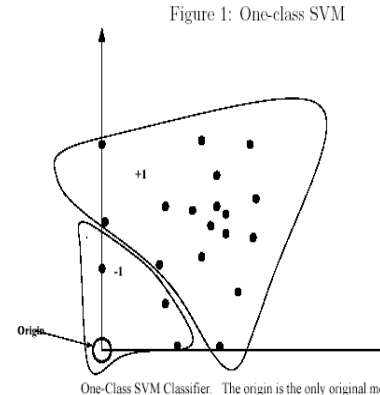
- Utiliza la hiperesfera mínima para encerrar la clase objetivo



OCSVM Resuelve el problema de optimización para encontrar la regla f con margen máximo

$$f(x) = \langle w, x \rangle + b$$

Si $f(x) < 0$, etiqueta x as anomalía



Detección de anomalías

Algoritmo Isolation Forest



- Isolation Forest es un algoritmo de detección de anomalías basado en ensemble.
- Funciona particionando recursivamente el conjunto de datos utilizando árboles de aislamiento.
- Se basa en que las anomalías son más fáciles de aislar y tienen caminos más cortos en los árboles



Detección de anomalías

Algoritmo Isolation Forest



1.- Construcción de árboles de aislamiento:

El algoritmo crea múltiples árboles de aislamiento (Isolation Trees o iTrees) para la construcción del bosque. Para cada árbol, se selecciona de manera aleatoria un subconjunto de los datos junto con un subconjunto de las características.

2.- Particionamiento de datos: En cada nodo del árbol, se elige aleatoriamente una característica y un valor de división entre el mínimo y máximo valor de esa característica. Los datos se dividen en dos nodos hijos según si son menores o mayores que el valor de división. Este proceso se repite recursivamente hasta que el punto de datos está aislado (es decir, es el único punto de datos en un nodo) o se alcanza una profundidad máxima del árbol.

Detección de anomalías

Algoritmo Isolation Forest

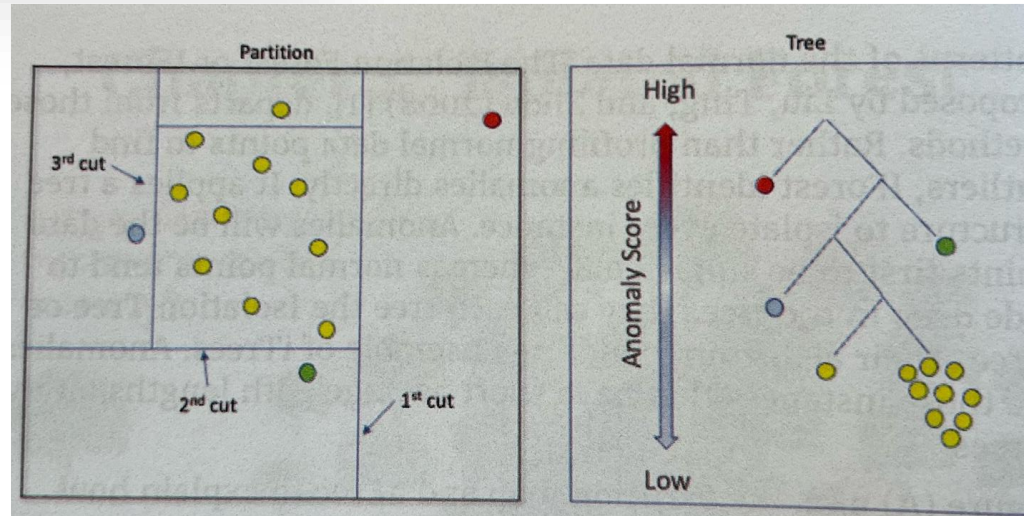


3.- Cálculo del puntaje de anomalía: La idea central es que las anomalías son más fáciles de aislar y, por lo tanto, serán aisladas en menos pasos (en otras palabras, tendrán una menor profundidad media en los árboles del bosque). El puntaje de anomalía de un punto de datos se basa en la profundidad promedio a la que se aísla en los diferentes árboles. Un puntaje más alto indica una mayor probabilidad de que el punto de datos sea una anomalía.

4.- Evaluación de resultados: Finalmente, se utilizan los puntajes de anomalía calculados para identificar las anomalías en el conjunto de datos. Los puntos de datos con puntajes de anomalía que superan un umbral definido se consideran anomalías.

Aislamiento de puntos con el algoritmo iTree

- El algoritmo iTree construye múltiples árboles de aislamiento para la creación del bosque.
- En cada nodo del árbol, se elige aleatoriamente una característica y un valor de división entre su valor mínimo y máximo.
- Los datos se dividen en dos nodos hijos según si son menores o mayores que el valor de división.
- Las anomalías son más fáciles de aislar y, por tanto, tendrán una menor profundidad media en los árboles del bosque.



Isolation Forest

- Esta matriz de datos muestra instancias. El objetivo de iForest es asignar una puntuación de outlier a cada una.
- Empieza seleccionando aleatoriamente un número de filas y columnas como (1), (2) y (3). Una instancia en al menos una tabla.
- Se construye un iTree en cada tabla para obtener puntuación de outliers. El primer corte para (1) es la sexta instancia, por tener valores muy diferentes. El Segundo sería la 4ª, etc...Para (2) también será la sexta el primer corte.
- Si hay N tablas, habrá N iTrees. Una instancia tiene hasta N puntuaciones. iForest calcula la media aritmética de las puntuaciones para obtener el score final.

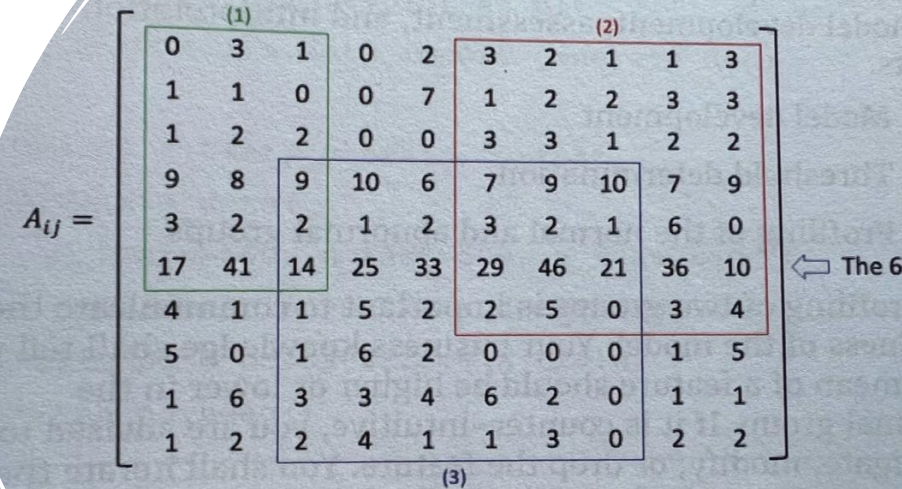


Figure (B): Isolation Forest

Isolation Forest

- Esta matriz de datos muestra instancias. El objetivo de iForest es asignar una puntuación de outlier a cada una.
- Empieza seleccionando aleatoriamente un número de filas y columnas como (1), (2) y (3). Una instancia en al menos una table.
- Se construye un iTree en cada tabla para obtener puntuación de outliers. El primer corte para (1) es la sexta instancia, por tener valores muy diferentes. El Segundo sería la 4ª, etc... Para (2) también será la sexta el primer corte.
- Si hay N tablas, habrá N iTrees. Una instancia tiene hasta N puntuaciones. iForest calcula la media aritmética de las puntuaciones para obtener el score final.

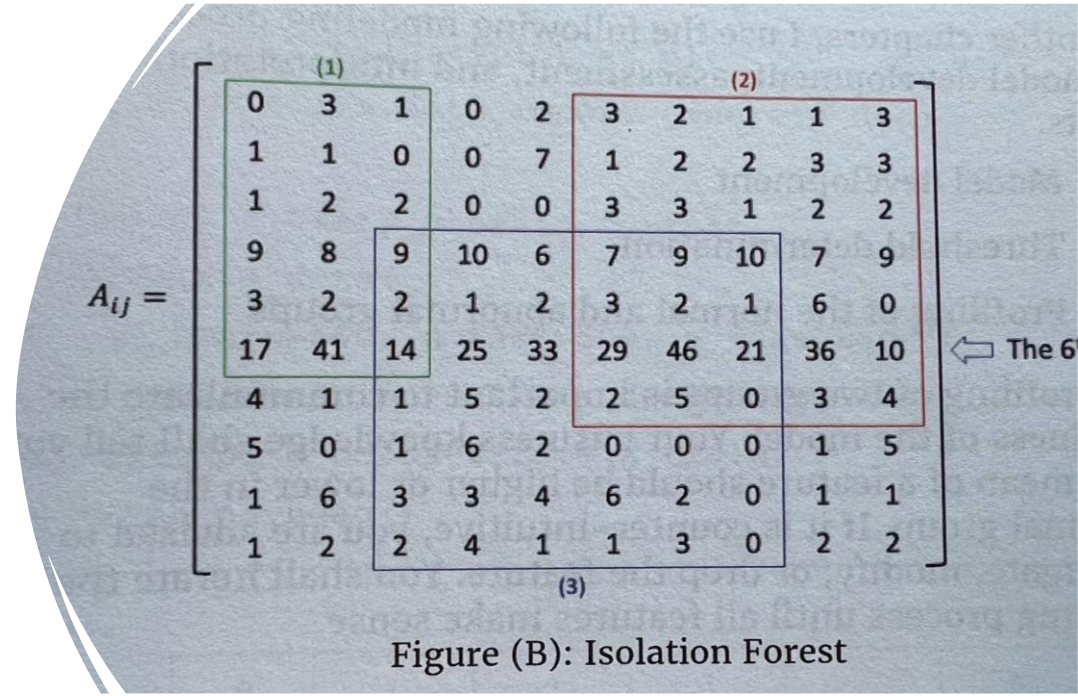


Figure (B): Isolation Forest